

# Proyecto #3 de Principios de Sistemas Operativos

- Instituto Tecnológico de Costa Rica
- Escuela de Ingeniería en Computación
- Principios de Sistemas Operativos
- Estudiantes:
  - Santiago Chaves Garbanzo, 2023047710
  - Sebastián Calvo Hernández, 2022099320
- Semestre I - 2026
- Fecha de Entrega: Domingo 28 de junio

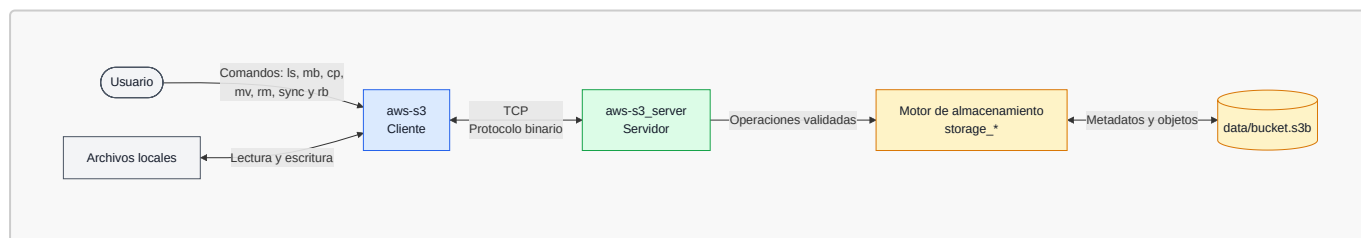
## Introducción

Este proyecto consiste en desarrollar una versión sencilla de un servicio de almacenamiento remoto inspirado en AWS S3. La idea principal es poder guardar, consultar, copiar, mover y eliminar archivos mediante un programa cliente, mientras que un programa servidor se encarga de procesar las solicitudes y mantener la información almacenada.

El proyecto fue desarrollado en C para un ambiente Linux. En nuestro caso se utilizó WSL para trabajar desde Windows con las herramientas y funciones de Unix. La comunicación entre el cliente y el servidor se realiza por medio de sockets TCP, por lo que ambos programas pueden ejecutarse de manera separada.

Además de implementar los comandos solicitados, se buscó que los archivos se transmitieran por partes y no se cargaran completos en memoria. Esto permite trabajar tanto con archivos de texto como con archivos binarios de mayor tamaño.

## Arquitectura general



## Descripción del problema

El problema consiste en simular un sistema de almacenamiento de objetos similar a AWS S3. En este tipo de almacenamiento no existen directorios reales dentro de los buckets. En su lugar, cada objeto se identifica mediante una clave. Por ejemplo, la clave `fotos/vacaciones/playa.jpg` contiene barras que el cliente puede mostrar como carpetas, aunque para el servidor sigue siendo una sola cadena.

El sistema se divide en dos programas. El programa `aws-s3` funciona como cliente y recibe comandos parecidos a los de AWS. El programa `aws-s3_server` recibe las solicitudes por la red, las valida y realiza las operaciones sobre los buckets.

Los comandos implementados permiten:

- Listar buckets, objetos y prefijos con `ls`.
- Crear buckets con `mb`.
- Copiar archivos entre una ruta local y S3, o entre dos ubicaciones S3, con `cp`.
- Mover archivos con `mv`.
- Eliminar objetos o prefijos completos con `rm`.
- Sincronizar directorios locales y prefijos remotos con `sync`.
- Eliminar buckets vacíos o forzar su eliminación con `rb`.

Cada bucket debe guardarse en un solo archivo. Al inicio de ese archivo se mantiene un bloque de directorio con la información necesaria para localizar los objetos. El resto del archivo contiene los datos de los objetos colocados de forma secuencial. Cuando se elimina o reemplaza un objeto, el espacio que ocupaba se registra para poder reutilizarlo después.

## Definición de estructuras de datos utilizadas

Para organizar el programa se utilizaron varias estructuras, cada una con una responsabilidad específica.

### Direcciones S3

La estructura `s3_uri_t` guarda el nombre del bucket y la clave del objeto que se obtienen de una dirección como `s3://bucket/carpeta/archivo.txt`. También registra si la dirección original terminaba en `/`, ya que esto ayuda a decidir si el destino representa un nombre exacto o un prefijo.

### Encabezado del protocolo

La estructura `protocol_header_t` representa el encabezado que acompaña cada mensaje entre el cliente y el servidor. Contiene:

- El código de la operación.
- Las banderas, por ejemplo `--recursive` o `--force`.
- El estado de la respuesta.
- La cantidad de bytes que contiene el mensaje.

El encabezado enviado por la red ocupa 24 bytes. Los valores numéricos se convierten a orden de red para evitar problemas si los programas se ejecutan en equipos con diferente representación de enteros.

Las estructuras `protocol_buffer_t` y `protocol_reader_t` se utilizan para construir y leer los mensajes. Con ellas se serializan cadenas y números sin enviar directamente las estructuras de C.

### Manifiestos de archivos

La estructura `manifest_entry_t` representa un archivo u objeto. Guarda su nombre, ruta local, tamaño, fecha de modificación y si corresponde a un prefijo. Varias entradas se almacenan en un `manifest_t`.

Los manifiestos se usan principalmente para recorrer directorios, mostrar listados y comparar el origen con el destino durante un `sync`.

### Objetos y espacios libres

Dentro del motor de almacenamiento, `object_entry_t` representa un objeto guardado en un bucket. Cada entrada contiene:

- La clave completa del objeto.
- La posición donde comienzan sus datos dentro del archivo `.s3b`.
- El tamaño del objeto.
- Su fecha de modificación.

La estructura `free_extent_t` guarda la posición y el tamaño de una región que quedó disponible después de eliminar o reemplazar un objeto.

Finalmente, `bucket_file_t` reúne el descriptor del archivo del bucket, la tabla de objetos, la lista de espacios libres, sus respectivos contadores y la posición donde termina el área de datos.

## Descripción de los componentes principales

### Cliente

El cliente se encuentra en `C/client.c`. Su primera tarea es interpretar el comando y las opciones escritas por el usuario. Después valida si cada ubicación es local o si utiliza el formato `s3://`.

Para operaciones recursivas, el cliente recorre los directorios locales y construye un manifiesto. Los comandos `cp`, `mv` y `sync` se realizan mediante una o varias operaciones simples enviadas al servidor. Por ejemplo, un movimiento se considera exitoso solamente si primero se pudo copiar el archivo; después de eso se elimina el origen.

### Servidor

El servidor está implementado en `C/server.c`. Este abre un socket, espera conexiones y procesa un cliente a la vez. Por cada solicitud recibe el encabezado, lee el contenido correspondiente y llama a la función adecuada del motor de almacenamiento.

El servidor también convierte los resultados a mensajes del protocolo. Si la operación falla, responde con un código de estado y una descripción corta del error.

### Protocolo de comunicación

Los archivos `C/protocol.c` y `H/protocol.h` contienen la lógica para serializar y recibir mensajes. El protocolo define operaciones para crear y eliminar buckets, listar objetos, subir, descargar, copiar, mover y eliminar objetos.

Las cadenas se envían como una longitud seguida por sus bytes. Los archivos se transmiten en bloques de 64 KiB, lo que evita reservar memoria según el tamaño completo del archivo.

### Motor de almacenamiento

El archivo `C/storage_file.c` contiene el almacenamiento persistente. Cada bucket corresponde a un archivo con extensión `.s3b`. El primer MiB está reservado para el encabezado, la tabla de objetos y la lista de espacios libres. Los datos comienzan después de este bloque.

El motor utiliza primer ajuste para buscar un espacio disponible. Si encuentra uno suficientemente grande, coloca ahí el nuevo objeto y conserva el sobrante como otro espacio libre. Si no encuentra espacio, agrega el objeto al final del archivo.

Cuando se reemplaza un objeto con otro del mismo tamaño, los nuevos datos se escriben en la misma posición. Si el tamaño cambia, se escribe la nueva versión al final y se libera la región anterior, de acuerdo con lo indicado en el enunciado. Los espacios libres contiguos se unen para disminuir la fragmentación.

## Utilidades comunes

Los módulos `common.c`, `uri.c` y `manifest.c` contienen funciones compartidas por el cliente y el servidor. Entre ellas se encuentran la lectura y escritura completa de datos, la conexión TCP, la validación de direcciones S3 y el recorrido de directorios locales.

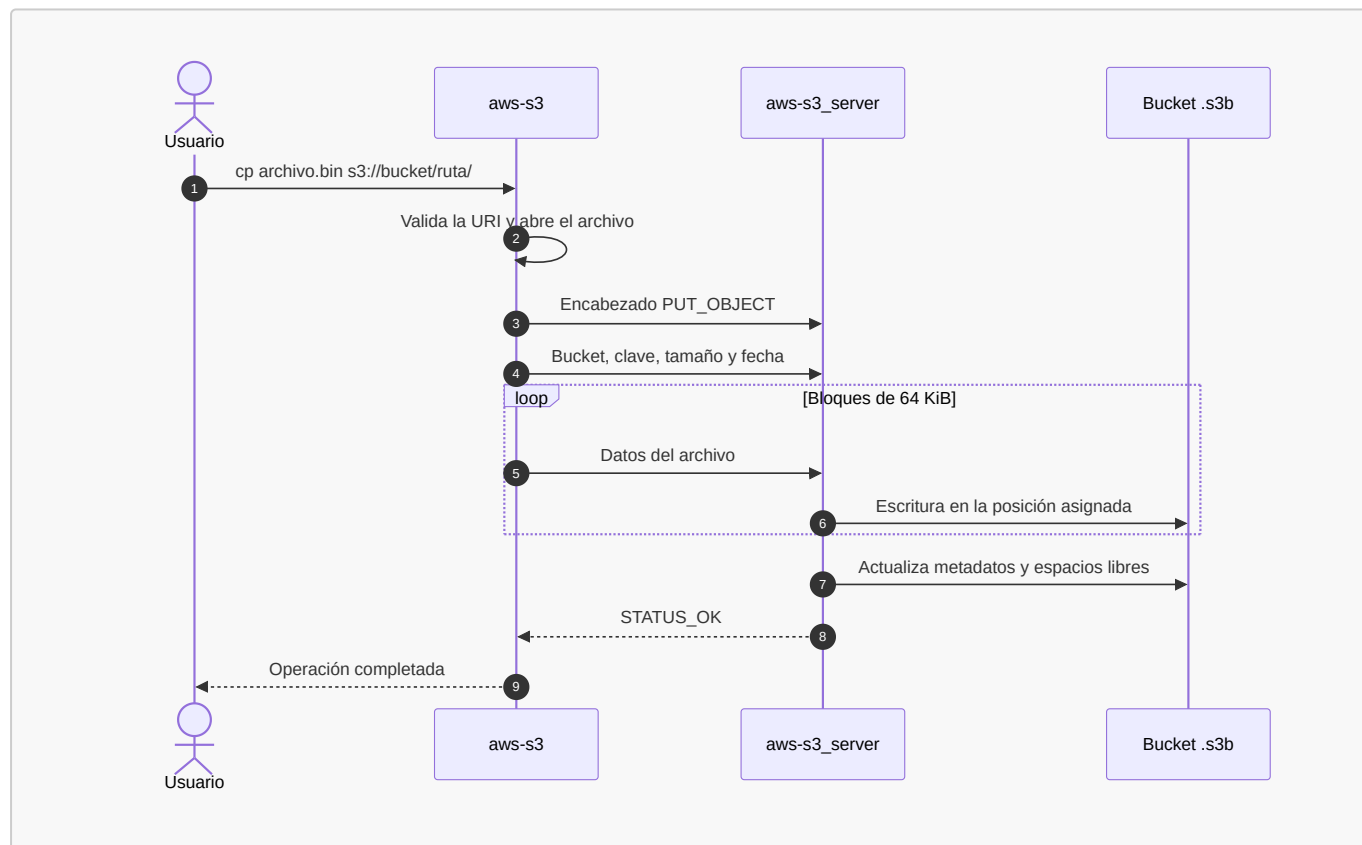
## Mecanismo de creación de archivos y comunicación con el servidor

Cuando se ejecuta `mb`, el cliente envía al servidor el nombre del bucket. El servidor crea un archivo llamado `<nombre>.s3b` dentro del directorio de datos. Al crearlo reserva un MiB para los metadatos y escribe un encabezado que identifica el formato y su versión.

Para subir un archivo, el cliente obtiene su tamaño y fecha de modificación, abre el archivo local y envía:

1. El encabezado del protocolo.
2. El nombre del bucket.
3. La clave del objeto.
4. El tamaño y la fecha de modificación.
5. El contenido del archivo en bloques de 64 KiB.

## Flujo de una subida



El servidor busca primero si el objeto ya existe y luego decide dónde guardar los datos. Para los objetos nuevos intenta reutilizar la primera región libre que tenga espacio suficiente. Si no existe una región adecuada, utiliza el final del archivo. Después actualiza el bloque de metadatos y fuerza su escritura al disco.

Para descargar un objeto se realiza el proceso contrario. El servidor consulta la tabla, obtiene la posición y el tamaño del objeto, y envía sus datos al cliente. El cliente crea los directorios locales que hagan falta, guarda el archivo y restaura su fecha de modificación.

Las copias entre buckets se realizan en el servidor utilizando la posición del objeto origen y el mismo mecanismo de escritura. En un movimiento, primero se completa la copia y solamente después se elimina el objeto original.

La conexión utiliza TCP para mantener el orden y asegurar que todos los bytes lleguen correctamente. Como una llamada a `read` o `write` puede procesar menos bytes de los solicitados, el proyecto utiliza funciones auxiliares que repiten la operación hasta completar el mensaje o detectar un error.

## Compilación

Requisitos: Linux o WSL, GCC y GNU Make.

```
make
```

Se generan:

- `bin/aws-s3`
- `bin/aws-s3_server`

## Ejecución

Inicie el servidor:

```
bin/aws-s3_server --host 127.0.0.1 --port 9000
```

Los buckets se guardan en `data/` por defecto. Puede elegir otra ubicación con `--data DIRECTORIO`.

En otra terminal:

```
bin/aws-s3 mb s3://ejemplo
bin/aws-s3 cp archivo.bin s3://ejemplo/
bin/aws-s3 ls s3://ejemplo/
bin/aws-s3 cp s3://ejemplo/archivo.bin ./descarga.bin
bin/aws-s3 rm s3://ejemplo/archivo.bin
bin/aws-s3 rb s3://ejemplo
```

El cliente acepta `--host` y `--port`, además de las variables `AWS_S3_HOST` y `AWS_S3_PORT`.

Comandos disponibles:

- `ls [s3://bucket/prefijo] [--recursive]`
- `mb s3://bucket`

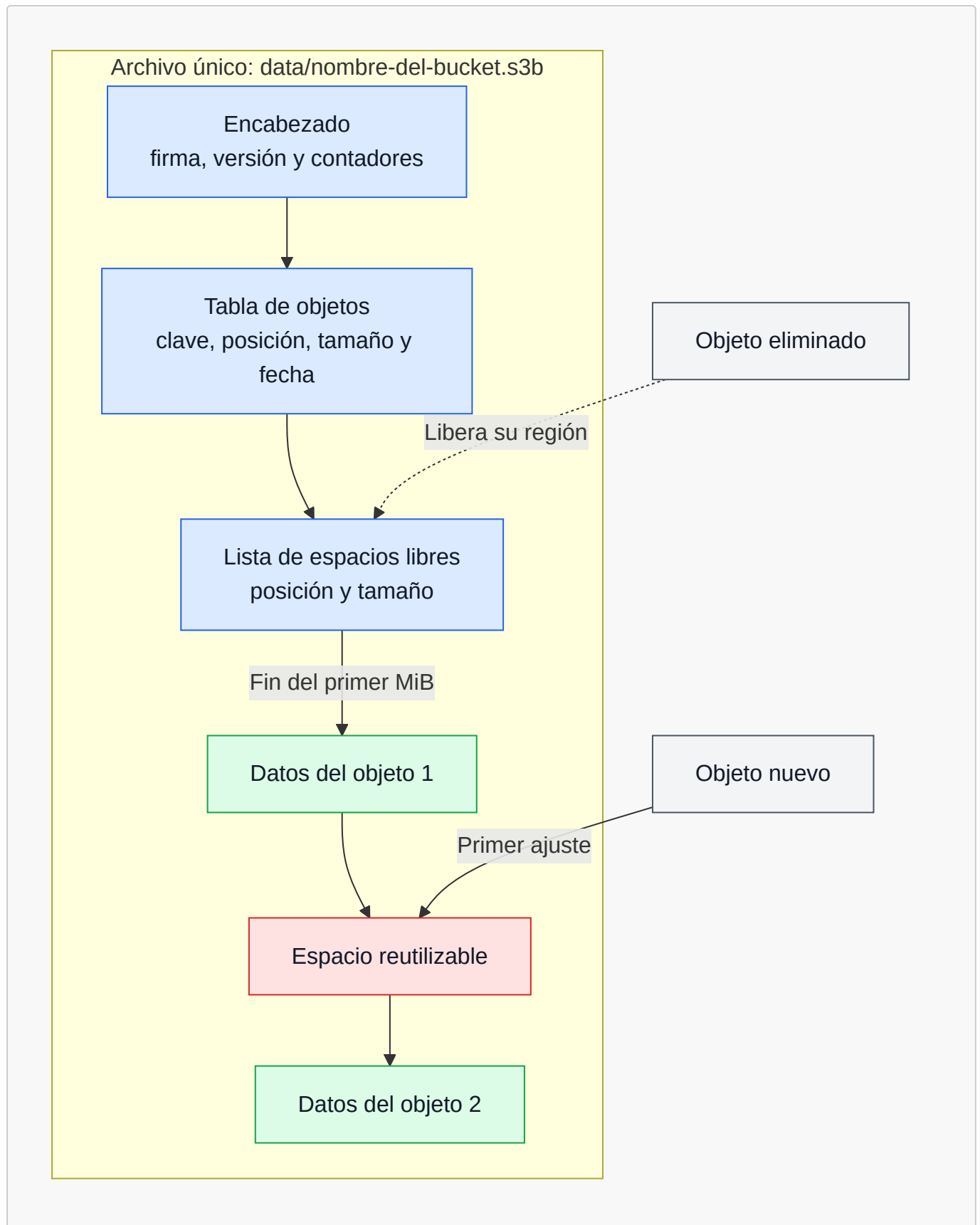
- `cp ORIGEN DESTINO [--recursive]`
- `mv ORIGEN DESTINO [--recursive]`
- `rm s3://bucket/clave [--recursive]`
- `sync ORIGEN DESTINO [--delete]`
- `rb s3://bucket [--force]`

No se siguen enlaces simbólicos. Los archivos se transmiten por streaming en bloques de 64 KiB.

## Almacenamiento

El primer MiB de cada archivo `.s3b` contiene el encabezado, la tabla de objetos y la lista de espacios libres. Los datos se almacenan después de ese bloque. Se utiliza primer ajuste para reutilizar espacio. La interfaz se describe en [docs/storage-interface.md](#) y el protocolo en [docs/protocol.md](#).

Distribución de un bucket



## Pruebas

Para dejar evidencia reproducible se creó la carpeta **pruebas/**, que contiene tres scripts:

- `sh pruebas/funcionales.sh`
- `sh pruebas/validacion.sh`
- `sh pruebas/rendimiento.sh`

Cada script crea su propio directorio temporal, inicia una instancia local del servidor, ejecuta las operaciones necesarias y elimina el entorno al finalizar.

## Pruebas funcionales

Las pruebas funcionales verificaron el flujo principal del sistema:

- Creación de bucket con `mb`.
- Subida de un archivo individual con `cp`.
- Subida recursiva de un directorio con `cp --recursive`.
- Listado de buckets y de objetos con `ls`.
- Descarga de un archivo y comparación con `cmp`.
- Eliminación de un prefijo con `rm --recursive`.
- Eliminación del último objeto y borrado del bucket con `rb`.

Resultado observado: todas las subpruebas anteriores finalizaron correctamente.

## Pruebas de validación

Las pruebas de validación se enfocaron en restricciones, persistencia e integridad:

- Rechazo de buckets duplicados.
- Rechazo de enlaces simbólicos como origen de `cp`.
- Rechazo de `rb` sobre un bucket no vacío sin `--force`.
- Descarga y comparación de un archivo de 80 KiB.
- Reinicio del servidor y nueva descarga del objeto para comprobar persistencia.
- Reemplazo de un objeto por otro del mismo tamaño y luego por otro de tamaño distinto para observar el tamaño del archivo bucket.

Resultados observados:

| Medición                                            | Resultado     |
|-----------------------------------------------------|---------------|
| Tamaño del bucket antes del reemplazo               | 1130502 bytes |
| Tamaño del bucket tras reemplazo del mismo tamaño   | 1130502 bytes |
| Tamaño del bucket tras reemplazo de distinto tamaño | 1130522 bytes |

Interpretación:

- El reemplazo con el mismo tamaño no aumentó el archivo del bucket.
- El reemplazo con tamaño distinto sí modificó el tamaño final del bucket.
- Los objetos siguieron disponibles después de reiniciar el servidor.

## Pruebas de rendimiento

Las pruebas de rendimiento se ejecutaron en el mismo equipo, usando conexión local `127.0.0.1`, para comparar dos escenarios:

- Un archivo grande de 16 MiB.
- Un conjunto de 200 archivos pequeños de 4 KiB cada uno.



Resultados observados:

| Medición                               | Resultado      |
|----------------------------------------|----------------|
| Subida de archivo grande               | 10 ms          |
| Descarga de archivo grande             | 9 ms           |
| Subida de 200 archivos pequeños        | 8410 ms        |
| Descarga de 200 archivos pequeños      | 16648 ms       |
| Tamaño final del bucket de rendimiento | 18644992 bytes |

Interpretación:

- La transferencia de un archivo grande resultó mucho más rápida que la de muchos archivos pequeños.
- El costo principal en el caso de archivos pequeños proviene de repetir la validación, serialización y envío de una operación por cada archivo.
- La descarga recursiva de archivos pequeños tardó más que la subida, lo cual es consistente con la creación repetida de rutas locales y aperturas de archivos en el cliente.

## Conclusiones

El proyecto permitió implementar un sistema de almacenamiento remoto inspirado en AWS S3 utilizando C, sockets TCP y un formato binario propio para la comunicación entre cliente y servidor. La separación entre cliente, servidor, protocolo y almacenamiento facilitó organizar la solución y asignar una responsabilidad clara a cada módulo.

Las pruebas funcionales y de validación mostraron que el sistema cumple con las operaciones principales solicitadas: creación y eliminación de buckets, copias en ambas direcciones, recorridos recursivos, persistencia después de reiniciar el servidor y manejo de errores esperados. También se comprobó que el motor de almacenamiento reutiliza espacio y mantiene los datos dentro de un único archivo por bucket, como pedía el enunciado.

En las pruebas de rendimiento se observó que el sistema trabaja mejor con archivos grandes que con muchos archivos pequeños. Esto confirma que, aunque la transmisión por bloques evita cargar archivos completos en memoria y funciona bien para datos grandes, el costo fijo de procesar cada archivo individual influye bastante cuando se realizan muchas operaciones pequeñas.

En general, el proyecto cumple con el objetivo de simular un servicio sencillo de almacenamiento de objetos, y además deja una base clara para futuras mejoras como concurrencia en el servidor, optimización de operaciones recursivas y una expansión del formato interno del bucket.